

Virtual Consensus in Delos

Balakrishnan et al. — OSDI 2020 — Facebook, Inc.

Gurjeet Kaur, Jia Hao Lin, Vandana Rawat, and Elizabeth Vargas

Roadmap

01

Motivation

Why is consensus hard to evolve in production?

02

Key Design Principles

Virtual Consensus

03

Delos System

Architecture and implementation at Facebook

04

Evaluation

Production upgrades, throughput, and overhead

05

Discussion

Trade-offs, limitations, and related work

The Problem: Consensus Systems Are Stuck

Deployed systems can't evolve

- ZooKeeper still runs a decade-old protocol (Zab)
- New research rarely reaches production
- Applications built from scratch around each system
- VMware CorfuDB uses shared acceptors; Facebook LogDevice uses flexible quorums

Hard to deploy & operate

No support for upgrading or migrating implementations without downtime — untenable for highly available services.

Hard to develop

Protocol embeds ordering + control plane (leader election, membership). New implementations must be rewritten from scratch.

Key Design Principles

01

Simplicity

Division of responsibility

- Loglets only handle ordering and durability
- Reconfiguration is removed entirely from the ordering layer.
- New protocol implementations much easier to build

02

Abstraction

Transparency in both directions

- Applications read and write to a plain log, and are unaware of what's underneath
- Loglets implement a standard interface, unaware of how they're chained

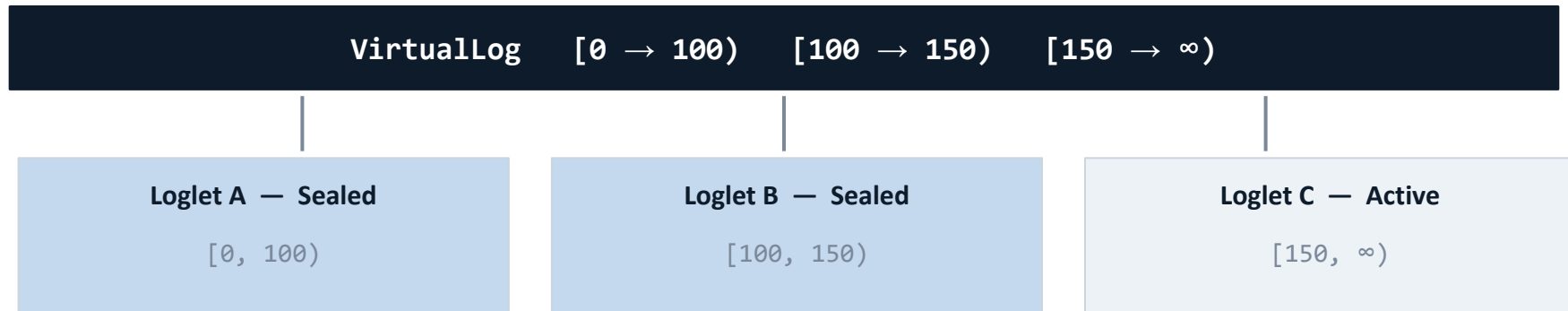
03

Modularity

Migration without downtime

- Different Loglet implementations can be swapped in and out of a live system without interrupting service
- Delos demonstrated this with a 10× latency gain and zero downtime

The VirtualLog: Chain Structure & Components



Client-Side Routing Layer

- Runs locally on the client.
- Translates virtual log positions to the correct Loglet's address space.
- Caches the chain locally, only contacts the MetaStore when the cache is stale, keeping it off the critical path.

MetaStore

- Centralized, versioned register storing the current chain configuration.
- Supports conditional writes: a write only succeeds if the version matches, preventing simultaneous reconfigurations.
- Single source of truth for the chain.

Loglets: Pluggable Ordering Protocols

ZKLoglet

ZooKeeper-based

Used to bootstrap Delos in its early deployment phase. Simple to operate (leverages existing ZooKeeper infrastructure) but limited in throughput and latency.

NativeLoglet

Custom Paxos · Primary production

The main Loglet in production today. Optimized for low latency. Switching to NativeLoglet from ZKLoglet delivered a 10× p99 latency reduction on a live system.

LDLoglet

Disaggregated · High throughput

Runs log storage on dedicated hardware, separate from the database. Eliminates I/O contention between the log and DB, boosting throughput by up to 33% on the same hardware.

StripedLoglet

Parallel stripes · Linear scale-out

Distributes appends across many LogServers in round-robin. 30 stripes across 30 servers reached 1M+ appends/sec — the highest single-log throughput in a non-emulated experiment.

Reconfiguration & Delos in Production

1

Seal the active Loglet

reconfigExtend blocks new appends and waits for all servers to acknowledge the seal.

2

Install the new Loglet

New config written to MetaStore via conditional write — immediately becomes the active segment.

3

Clients discover the change

Stale clients re-read the MetaStore and resume appends on the new Loglet automatically.

4

Remove the old segment

reconfigTruncate drops the sealed Loglet once entries are replicated — no seal, no latency spike.

Applications

Delos · Table API · ZooKeeper API

VirtualLog + MetaStore

Loglets (ZK | Native | LD | Striped)

1.5B

transactions/day

8 mo.

time to production

Evaluation: Production Upgrade Without Downtime

April 2, 2019 — live switch from ZKLoglet to NativeLoglet on a production Delos instance

10×

latency reduction

p99, multi-gets & indexed queries

5×

improvement

p99 for multi-puts

0

downtime

service availability uninterrupted

What happened:

- reconfigExtend sealed ZKLoglet and installed NativeLoglet — latency drop was immediate
- reconfigTruncate removed old ZKLoglet — no multi-put disruption (no seal needed for truncation)
- Only latency spikes: indexed queries during reconfiguration — availability was never interrupted
- 4 months later: re-upgraded again for an additional 10× improvement

Evaluation: Throughput Scaling

Virtual consensus does not sacrifice throughput — and enables dynamic scaling

10×

Throughput gain

Switching to disaggregated LDLoglet at 150K ops/s without downtime — separate log and DB resources eliminate I/O contention.

33%

More throughput

Disaggregated LDLoglet vs. converged NativeLoglet on benchmark HW — log/DB on separate machines removes contention.

1M+

Appends/sec (Striped)

StripedLoglet scales linearly: 30 stripes across 30 LogServers each providing 30K IOPS with p99 < 75ms.

Cost of Virtualization

Virtualization overhead is low — mostly on the async/RPC path, not the critical ordering path

Critical path overhead (p99, production)

readNext

~6 μ s

Synchronous pass-through

append

100–150 μ s

Async future-based API

checkTail

100–150 μ s

Async future-based API

reconfigExtend

< 10 ms

Seal + MetaStore write

vs. ZooKeeper (apples-to-apples)

At 1 GB — Delos+NativeLoglet matches ZooKeeper throughput

At 100 GB — ZooKeeper provides 30K+ puts/sec; Delos matches with p99 < 15ms

At 1 TB — ZooKeeper stores state in memory and struggles; Delos scales via RocksDB and continues to serve the same SLA

ZooKeeper runs out of memory; Delos does not

Discussion: Benefits and Limitations

Benefits

- Incremental deployment: production in 8 months
- Online upgrades: swap protocols without downtime
- Dynamic performance tuning at runtime
- Simple protocols are easier to build and verify
- Entry deletion and remapping via VirtualLog metadata

Limitations

- Fate sharing in converged mode — DB fragility linked to log fragility (mitigated by disaggregation)
- No support for speculative protocols, approximate synchrony, or commutative operations
- Converged ↔ disaggregated switching is currently manual (operator-driven)
- Only fail-stop consensus — not Byzantine fault tolerant

Related Work

How this paper differs from existing systems

Traditional Consensus Systems

- Raft, ZAB, Viewstamped Replication
- Strong consistency
- Hard to upgrade (tightly coupled design)

Shared Log Systems

- CORFU, LogDevice, Kafka
- Abstract consensus via logs
- Still monolithic → no runtime switching

Key Limitation

- Cannot change consensus without downtime
- No plug-and-play flexibility
- Innovation ≠ production

This Paper:

Virtualizes consensus → switch protocols dynamically without downtime

Conclusion

Virtualize consensus:	decouple the ordering mechanism (Loglet) from the reconfiguration / control plane (VirtualLog)
Loglets are simple:	no leader election, no reconfiguration — just order and store; swap any Loglet without touching applications
Production evidence:	8 months to deploy at Facebook; 10× latency reduction with zero downtime via live protocol swap
Generalizable:	any shared-log-based system can migrate to virtual consensus; new protocols become first-class citizens